

Laboratorio di Programmazione

Lezione 10: Mappe, Strutture, Puntatori

1	MAPPE - Qual è l'output?	3
2	Qual è l'output?	4
3	Valori distinti	5
4	Simboli unici	6
5	Conteggio	7
6	Istogramma a barre orizzontali	8
7	Istogramma a barre orizzontali (2)	9
8	Lettura mappa	10
9	Inversione mappa	11
10	TABELLA: confronto tra mappe e slice	12
11	STRUTTURE - Qual è l'output?	12
12	Qual è l'output?	14
13	Matrici	15
14	Copia superficiale	15
15	Copia profonda	15
16	Rubrica	16
17	PUNTATORI - Qual è l'output?	18
18	Qual è l'output?	18
19	Qual è l'output?	19
20	Trova l'errore	20
21	Trova l'errore	20

22 Scambio valori.....	21
23 Moltiplicazione array.....	21
24 Puntatori a variabili locali	21
25 Clonazione struttura	21
26 Matrice	22
27 Estrazione puntatori	22
28 Array dinamico	22
29 Lista Dinamica.....	23

1 MAPPE - Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var m map[string]int
7     fmt.Println(m)
8     m = make(map[string]int)
9     for _, s := range []string{"questo", "è", "un", "test"} {
10         m[s] = len([]rune(s))
11     }
12     for k, v := range m {
13         fmt.Println(k, "->", v)
14     }
15     fmt.Println()
16     for k := range m {
17         fmt.Println(k, "->", m[k])
18     }
19 }
```

2 Qual è l'output?

```
1 package main
2
3 import (
4     "fmt"
5 )
6
7 var (
8     nominativi = map[string]string{"023314944": "Mario Rossi",
9         "024158685": "Carlo Bianchi", "026424971": "Giuseppe Verdi",
10        "0269001634": "Carlo Bianchi", "026691369": "Mario Rossi",
11        "0248704925": "Carlo Bianchi", "023554756": "Giuseppe Verdi"}
12 )
13
14 func main() {
15     numeriTelefonici := make(map[string][]string)
16     for k, v := range nominativi {
17         numeriTelefonici[v] = append(numeriTelefonici[v], k)
18     }
19     for k, v := range numeriTelefonici {
20         fmt.Printf("Nominativo: %v\nNumero telefonico: %v\n\n", k, v)
21     }
22 }
```

3 Valori distinti

Scrivete un programma che legge da standard input una sequenza di coppie chiave-valore, una per riga, nel formato

K V

Decidete voi il tipo dati (es. K stringa e N intero). Il programma deve poi stampare il numero di valori distinti che appaiono nella mappa. Non usate slice o array, ma solo mappe.

Esempio d'esecuzione

```
$ go run valori.go
```

```
aaa 5
```

```
bbb 5
```

```
ccc 6
```

```
ddd 4
```

```
eee 6
```

```
valori distinti: 3
```

4 Simboli unici

Scrivere un programma che legge da riga di comando una stringa (potete anche immettere una stringa senza spazi, racchiudendola fra singoli apici). Facendo opportuno uso delle mappe, il programma deve stampare l'elenco dei simboli che appaiono nella stringa (ogni simbolo va quindi stampato *una sola volta*). L'ordine di stampa non è rilevante.

Esempio d'esecuzione:

```
go run uniq.go 'ambarabacicicoco!'
r c a ! i b o m
```

5 Conteggio

Scrivere un programma che legge da riga di comando una sequenza di numeri interi di lunghezza arbitraria. Lo stesso intero può apparire più volte. Il programma deve stampare, per ogni intero nella sequenza, il suo numero di occorrenze (l'ordine in cui sono stampati non è importante).

Esempio d'esecuzione:

```
$ go run conta.go 0 0 1 0 1 2 1 1 0 1
valore  occorrenze
0        4
2        1
1        5
```

6 Istogramma a barre orizzontali

Scrivere un programma che legge da standard input un testo su più righe, terminato da EOF. Il programma deve poi stampare un istogramma a barre orizzontali che mostri il numero di occorrenze di ogni lettera che appare nel testo. Il programma deve implementare le seguenti funzioni, di ovvio significato:

- LeggiTesto() string
- Occorrenze(s string) map[rune]int

Ricordate che potete usare la funzione `unicode.IsLetter()`.

Esempio d'esecuzione:

```
$ go run istogrammaV1.go
TESTO di prova
disposto su più righe!
Istogramma:
i: ****
a: *
h: *
d: **
r: **
g: *
e: *
p: ***
s: ***
t: *
u: *
o: ***
E: *
S: *
O: *
v: *
ù: *
T: **
```


7 Istogramma a barre orizzontali (2)

Adattare la soluzione dell'esercizio 6 in modo da stampare le barre in ordine crescente di codice Unicode. Per ordinare una slice di rune potete usare la seguente funzione.

```
1 func SortRunes(a []rune) {
2     for i:=0; i<len(a)-1 ;i++){
3         indiceMin := i
4         for j := i + 1; j<len(a); j++ {
5             if a[indiceMin] > a[j] {
6                 indiceMin = j
7             }
8         }
9         a[i], a[indiceMin] = a[indiceMin], a[i]
10    }
11 }
```

Esempio d'esecuzione:

```
$ go run istogramma.go
Ciao,
come stai?
Occorrenze:
C: *
a: **
c: *
e: *
i: **
m: *
o: **
s: *
t: *
```

8 Lettura mappa

Scrivere un programma che legga da standard input una lista di righe nel formato

`S N`

dove `S` è una stringa senza spazi ed `N` un intero. Notate che la stessa stringa può apparire in più righe, associata a diversi interi o anche allo stesso intero. In programma deve quindi creare una mappa in cui `S` è associata alla lista degli interi `N` che gli appaiono insieme in qualche riga. Il programma deve poi stampare le stringhe `S` in ordine lessicografico (vedete `sort.Strings()`), ognuna seguita dai suoi interi in ordine crescente (vedete `sort.Ints()`). Il programma deve implementare le seguenti funzioni:

- `LeggiRighe()` (`S []string, N []int`) che restituisce due slice i cui elementi *i*-esimi sono la stringa e l'intero presente sulla *i*-esima riga (ovviamente *i* parte da 0).
- `CreaMappa(S []string, N []int) map[string][]int` che prende l'output di `LeggiRighe` e crea la mappa descritta sopra.

Esempio d'esecuzione:

```
$ go run leggimappa.go
```

```
aaa 5  
bbb 13  
ccc 4  
aaa 7  
ccc 4  
bbb 2  
aaa 15
```

```
aaa: 5 7 15  
bbb: 2 13  
ccc: 4
```

9 Inversione mappa

Scrivete una funzione:

```
InvertiMappa(M map[string]int) W map[int][]string
```

che calcola W, la mappa inversa di M. Questo significa che, per ogni *valore* n che appare in M, la slice W[n] è la lista di tutte le chiavi che, in M, hanno n come valore.

Applicate la funzione all'esercizio precedente. Per semplicità, iniziate assumendo che ogni chiave s appaia solo una volta in input, e adattando CreaMappa in modo che ritorni map[string]int invece di map[string][]int. Stampate poi la lista di valori, in ordine crescente, ciascuno con le sue chiavi, in ordine lessicografico.

Esempio d'esecuzione

```
$ go run inverti.go
aaa 5
bbb 10
ccc 15
ddd 5
```

valore	chiavi
5	aaa ddd
10	bbb
15	ccc

Modificate poi la soluzione in modo da ammettere ripetizioni di chiave. In questo caso CreaMappa deve ritornare map[string][]int, come nella versione originale, e dovrete quindi cambiare InvertiMappa in modo che prenda in input una map[string][]int.

Esempio d'esecuzione

```
$ go run inverti.go
aaa 5
bbb 10
ccc 15
ddd 5
aaa 10
bbb 5
```

valore	chiavi
5	aaa bbb ddd
10	aaa bbb
15	ccc

10 TABELLA: confronto tra mappe e slice

Aspetto	Slice	Mappa
Cosa memorizza	Collezione ordinata sequenziale	Associazione chiave-valore
Reference type	Sì	Sì
Valore zero	nil (ma puoi fare append)	nil (scrivere causa panic!)
Inizializzazione	<code>s := make([]int, 10)</code> <code>s := []int{}</code>	<code>m := make(map[string]int)</code> <code>m := map[string]int{}</code>
Accesso elementi	Per indice numerico: <code>s[0]</code> , <code>s[2*k+5]</code>	Per chiave: <code>m["nome"]</code> , <code>m[42]</code>
Tipo indice/chiave	Solo int, da 0 a <code>len(s)-1</code>	Qualsiasi tipo comparabile
Elemento inesistente	Panic (indice fuori range)	No panic, restituisce valore zero
Controllo esistenza	Controlla se indice in $0, \dots, \text{len}(s)-1$	<code>val, ok := m[chiave]</code>
Aggiunta elementi	<code>s = append(s, elemento)</code>	<code>m[chiave] = valore</code>
Rimozione elementi	<code>s = append(s[:i], s[i+1:]...)</code>	<code>delete(m, chiave)</code>
Iterazione	<code>for i, v := range s</code>	<code>for k, v := range m</code>
Ordine iterazione	Sequenziale: $i = 0, 1, \dots, \text{len}(s)-1$	Arbitrario
Lunghezza	<code>len(s)</code> , numero di elementi	<code>len(m)</code> , numero di chiavi
Capacità	<code>cap(s)</code>	Non applicabile
Copia	<code>copy(dest, src)</code> funzione built-in	A mano, usando <code>for range</code>
Crescita dinamica	No (va usato append)	Sì
Uso tipico	Liste, array dinamici, anche multidimensionali	Dizionari, insiemi, chiavi non numeriche o non in un intervallo $\{0, \dots, n-1\}$ con n piccolo.
Esempio pratico	<code>[]int{1, 2, 3, 4}</code>	<code>map[string]int{"a": 1, "b": 2}</code>

11 STRUTTURE - Qual è l'output?

```
1 package main
2 import "fmt"
3
4 type Persona struct {
5     Nome string
6     Cognome string
7 }
```

```
8
9 func main() {
10     var p1 Persona
11     p1 = Persona{"Rick", "Sanchez"}
12
13     var p2 Persona
14     p2.Nome = "Jerry"
15     p2.Cognome = "Smith"
16
17     p3 := Persona{Nome: "Morty"}
18
19     p2 = p3
20     fmt.Println(p1, p2)
21 }
```

12 Qual è l'output?

```
1 package main
2 import "fmt"
3
4 type Persona struct {
5     Nome string
6     Cognome string
7 }
8
9 func main() {
10     anagrafica := []Persona{
11         {"Rick", "Sanchez"},
12         {"Morty", "Smith"},
13         {"Jerry", "Smith"},
14         {"Summer", "Smith"},
15         {"Beth", "Smith"}}
16     CambiaPrimoCognome(anagrafica)
17     CambiaCognome(anagrafica[0])
18     fmt.Println(anagrafica[0])
19 }
20
21 func CambiaPrimoCognome(a []Persona) {
22     a[0].Cognome = "Martinez"
23 }
24
25 func CambiaCognome(p Persona) {
26     p.Cognome = "Gomez"
27 }
```

13 Matrici

Definite un tipo `Matrice` che rappresenta una matrice di valori reali $A \in \mathbb{R}^{n \times m}$. La struttura deve avere campi per memorizzare: il numero n di righe, il numero m di colonne, e ovviamente i valori. Scrivete una funzione

```
LeggiMatrice() Matrice
```

che legge una matrice da standard input, riga per riga (i valori di n ed m vanno dedotti dall'input). Adattate gli esercizi delle *slice di slice* in modo da usare questa struttura.

14 Copia superficiale

Adattate l'esercizio precedente aggiungendo una funzione `Copia(M Matrice) Matrice` che restituisce una copia di M . Verificate che, modificando gli elementi della copia, viene modificata anche M .

15 Copia profonda

Adattate l'esercizio precedente in modo che `Copia(M Matrice) Matrice` restituisca una copia *profonda*, cioè i cui dati sono memorizzati in locazioni di memoria *diverse* da quelle della matrice originale. Verificate che, modificando gli elementi della copia, M non viene modificata.

16 Rubrica

Dovete implementare una rubrica di contatti, ognuno con nome, cognome, e indirizzo. Non potranno esistere due contatti con stesso nome e cognome. Definite i seguenti tipi di dati:

- Indirizzo: struttura con campi via, città di tipo string, e civico di tipo uint;
- Contatto: struttura con campi cognome, nome di tipo string, e indirizzo di tipo Indirizzo;
- Rubrica: mappa con chiavi string e valori Contatto.

La chiave associata ad un contatto è formata dalla concatenazione di nome e cognome, separati da un pipe |, dove eventuali spazi (es. nel caso di secondo nome) sono sostituiti da punti. Per esempio un contatto con nome “Roberto Fitzgerald” e cognome “Cipollotti Viendalmare” darà questa chiave:

```
Roberto.Fitzgerald|Cipollotti.Viendalmare
```

Implementate le seguenti funzioni, di ovvio significato:

- CreaIndirizzo(via, città string, civico uint) Indirizzo
- CreaContatto(nome, cognome string, indirizzo Indirizzo) Contatto
- ChiaveContatto(c Contatto) string
- StampaContatto(c Contatto)
- StampaRubrica(r Rubrica)

Il programma dovrà poi leggere, da standard input, una sequenza di *comandi* di inserzione, eliminazione, aggiornamento, o stampa della rubrica. Il formato dei comandi è come segue:

<i>inserimento contatto</i>	I;cognome;nome;via;civico;città
<i>aggiornamento contatto</i>	A;cognome;nome;via;civico;città
<i>eliminazione contatto</i>	E;cognome;nome
<i>stampa rubrica</i>	S

Esempio d'esecuzione:

```
$ cat comandi.txt
I;Mario;Rossi;Via Celoria;18;Milano;
I;Mario;Rossi;Via Celoria;18;Milano;
S
I;Elena;Bianchi;Via Celoria;18;Milano;
S
E;Mario;Rossi
S
A;Elena;Bianchi;Via Festa del perdono;7;Milano;
S
```

```
$ go run rubrica.go < comandi.txt
Rubrica:
[1] - Mario Rossi: Via Celoria 18, Milano
```


Rubrica:

[1] - Mario Rossi: Via Celoria 18, Milano

[2] - Elena Bianchi: Via Celoria 18, Milano

Rubrica:

[1] - Elena Bianchi: Via Celoria 18, Milano

Rubrica:

[1] - Elena Bianchi: Via Festa del perdono 7, Milano- Tel. 02503111

17 PUNTATORI - Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var a, b, c int
7     var ptr *int
8     a, b, c = 10, 15, 20
9     ptr = &a
10    *ptr += 10
11    a += 10
12    ptr = &b
13    *ptr += 10
14    *ptr = c
15    *ptr += 10
16    fmt.Println(a, b, c)
17 }
```

18 Qual è l'output?

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var a, b = 15, 20
7     f(&a, &b)
8     fmt.Println(a, b)
9 }
10 func f(x, y *int) {
11     *x, *y = *y, *x
12 }
```

19 Qual è l'output?

```
1 package main
2
3 import (
4     "fmt"
5     "strings"
6 )
7
8 type Persona struct {
9     Nome, Cognome string
10 }
11
12 func main() {
13
14     p := &Persona{"Rick", "Sanchez"}
15
16     f(*p)
17     fmt.Println(p)
18     g(p)
19     fmt.Println(p)
20 }
21
22 func f(p Persona) {
23     p.Nome, p.Cognome = strings.ToUpper(p.Nome), strings.ToUpper(p.Cognome)
24 }
25
26 func g(p *Persona) {
27     p.Nome, p.Cognome = strings.ToUpper(p.Nome), strings.ToUpper(p.Cognome)
28 }
```

20 Trova l'errore

Correggete il programma in modo che stampi 20 100.

```
1 package main
2 import "fmt"
3
4 func main() {
5     var a, b int = 10, 20
6     var ptr *int
7     ptr = a
8     *ptr = *ptr + b
9     fmt.Println(a, b)
10 }
```

21 Trova l'errore

Correggete il programma in modo che stampi 20 100.

```
1 package main
2
3 import "fmt"
4
5 func main() {
6     var a, b int = 10, 20
7     var ptr *int
8     *ptr = 50
9     ptr = &a
10    *ptr += b
11    fmt.Println(a, b)
12 }
```

22 Scambio valori

Scrivete una funzione `Swap`, di opportuna segnatura, che prenda in input due interi e li scambi. Per esempio, il seguente codice, opportunamente completato, dovrà stampare 3 24:

```
1  var a, b int 24, 3
2  Swap(.....)
3  fmt.Println(a, b)
```

23 Moltiplicazione array

Scrivete una funzione `Moltiplica`, di opportuna segnatura, che prende un puntatore ad array di 5 valori reali, ed un ulteriore reale p , e moltiplica ogni elemento dell'array per p . Per esempio il seguente codice, opportunamente completato, dovrà stampare [1.5 3 4.5 6 7.5]:

```
1  var v [5]float64 = [5]float64{1, 2, 3, 4, 5}
2  Molt(....., 1.5)
3  fmt.Println(v)
```

24 Puntatori a variabili locali

Scrivete una funzione `Crea42() *int` che restituisce il puntatore ad una variabile intera, dichiarata nella funzione, con valore 42.

25 Clonazione struttura

Definite un tipo dati `Punto struct {X, Y float64}`. Implementate poi una funzione `Clona(p *Punto) *Punto` che crea una copia di `p`. Assicuratevi che, modificando i valori della copia, l'originale non subisca modifiche.

26 Matrice

Definite una variabile `M [10][10] int`. Implementate poi una funzione `Set()` di opportuna segnatura che vi permetta di modificare il valore di `M[i][j]`. Testate.

27 Estrazione puntatori

Scrivete una funzione `Puntatori(S []int) []*int` che per ogni elemento in `S` ne produce il puntatore. Testatela.

28 Array dinamico

Definite un tipo dati `DynArray struct {data [] int}`. Implementate poi le seguenti funzioni:

- `Add(x int)` che aggiunge un elemento all'array.
- `Get(i int) *int` che restituisce il puntatore all'elemento `i` dell'array.

Testatele.

29 Lista Dinamica

Definite il tipo dati:

```
1 type Node struct {  
2     val int  
3     next *Node  
4 }
```

Notate che il campo `Next` è un puntatore ad una variabile di tipo `Node` stesso. Una *lista* è un insieme di nodi, in cui il primo punta al secondo, il secondo al terzo, e così via. La lista è identificata dal primo nodo, detto *testa* (head). Implementate le seguenti funzioni per gestire una lista:

- `Add(lista *Node, val int)` che accoda alla lista un nuovo nodo con valore `val`. Il nuovo nodo deve, naturalmente, essere puntato dall'ultimo nodo nella lista attuale.
- `Stampa(lista *Node)` che stampa tutti i valori nella lista, uno per riga.

Testate la loro correttezza, invocando qualche volta `Add` e verificando che `Print` stampi tutti i valori aggiunti.